

1 DN1: SOR iteracija za razpršene matrike

Avtor: Patrik Schembri

V nalogi se ukvarjamo s SOR iteracijo na razpršenih matrikah. Matrika hrani svoje vrednosti v matrikah V in I , ki sta velikosti $n \times m$. Pri tem velja:

$$V(i, j) = A(i, I(i, j))$$

```
using Domaca01, LinearAlgebra, Plots
```

Definiramo nov tip - Razpršeno matriko A .

```
V = [[1, 2], [3, 4, 5]]  
I = [[1, 2], [1, 2, 3]]  
A = RazprsenaNatrika(V, I)
```

```
Domaca01.RazprsenaNatrika([[1, 2], [3, 4, 5]], [[1, 2], [1, 2, 3]])
```

Ker gre za poseben tip matrike moramo uvesti nove funkcije za naslenje funkcionalnosti:

Funkcija `getindex`

```
ix = getIndex(A, 1, 2)
```

2

Funkcija `setindex`

```
setindex!(A, 5, 1, 2)  
A
```

```
Domaca01.RazprsenaNatrika([[1, 5], [3, 4, 5]], [[1, 2], [1, 2, 3]])
```

Funkcija `firstindex`

```
first = firstindex(A, 1)
```

1

Funkcija `lastindex`

```
last = lastIndex(A, 2)
```

3

1.1 SOR iteracija

Sedaj lahko izvedemo Successive over-relaxation oz. SOR iteracijo. SOR iteracija se uporablja predvsem za reševanje sistemov $Ax = b$. Pri tem upošteva tudi relaksacijski vektor ω , ki je po navadi v mejah med 0 in 2. Iteracija deluje po naslednji formuli:

$$x_i^{k+1} = (1 - \omega) * x_i^k + \omega / A_{(i,i)} * (b_i - \sum_{(j < i)} A_{(ij)} * x_j^{k+1} - \sum_{(j > i)} A_{(ij)} * x_j^k)$$

Iteracija se konča ko dosežemo konvergenco po naslednjem kriteriju, kjer za toleranco vzamemo zelo majhno vrednost (npr. $1e-10$): $|Ax(k) - b| \infty < \text{toleranca}$ Računamo SOR iteracijo za razpršeno matriko A in vektor b za ω 1.25. x_0 predstavlja začetni približek.

```

V = [[2.0], [1.0, 1.0], [3.0]]
I = [[1], [2, 3], [3]]
A = RazprsenoMatrika(V, I)

b = [1.0, 4.0, 2.0]
x0 = [0.0, 0.0, 0.0]
omega = 1.25

x, iter = SOR(A, b, x0, omega)

([0.499999999999999954525, 3.3333333333333332696684, 0.66666666666666660603], 20)

```

Izvedemo vlaganje grafa na ravnino / prostor s fizikalno metodo. Če so (x_i, y_i, z_i) koordinate vozlišč grafa v prostoru, potem vsaka koordinata posebej zadošča enačbam

$$-st(i)x[i] + \sum_{j \in N(i)} x[j] = 0 - st(i)y[i] + \sum_{j \in N(i)} y[j] = 0 - st(i)z[i] + \sum_{j \in N(i)} z[j] = 0$$

Funkcija `ustvariRazprsenoMatriko` pretvori podane informacije o grafu (vozlišča, robovi in uteži) v razpršeno matriko, ki jo lahko nato uporabimo pri izvajanju numeričnih metod (npr. SOR iteracije).

Funkcija `optimalnaOmega` izračuna optimalno vrednost parametra omega (relaksacijski faktor), ki minimizira število iteracij pri SOR metodi. Prav tako vrne graf hitrosti konvergence glede na različne vrednosti omega.

Funkcija `sistemLaplace` pripravi sistem linearnih enačb na podlagi grafa, ki ga predstavljajo vozlišča in robovi. Najprej funkcija za dani graf zgradi Laplaceovo matriko tipa RazprsenoMatrika. Diagonalni elementi predstavljajo stopnjo vozlišča (število povezav), nediagnostni pa -1 za vsako sosedstvo. Nato se iz matrike odpravijo vrstice in stolpci, ki pripadajo vnaprej določenim (fiksni) vozliščem. Rezultat je zmanjšana Laplaceova matrika Lred in dva vektorja desne strani bx in by, ki predstavljata vpliv fiksnih vozlišč na sistem v x in y smeri.

Primer 1: Preverjanje pravilnosti metode z znanim rezultatom (LU razcep) Tu uporabimo sistem, kjer poznamo točen rezultat, da lahko preverimo pravilnost funkcije `optimalnaOmega`.

```

vozlisca = [1, 2, 3]
robovi = [(1, 1), (2, 2), (2, 3), (3, 3)]
vrednosti = [2.0, 1.0, 1.0, 3.0]
A = ustvariRazprsenoMatriko(vozlisca, robovi, vrednosti)

b = [1.0, 4.0, 2.0]
x0 = [0.0, 0.0, 0.0]

plot, minIteracij, minOmega, rez = optimalnaOmega(A, b, x0, 1)

(Plot{Plots.GRBackend() n=1}, 10.0, 0.9473684210526315, [0.4999999999999999184
5, 3.333333333333333523623, 0.6666666666666665579])

```

Optimalna vrednost omega

minOmega

0.9473684210526315

Minimalno število iteracij

minIteracij

10.0

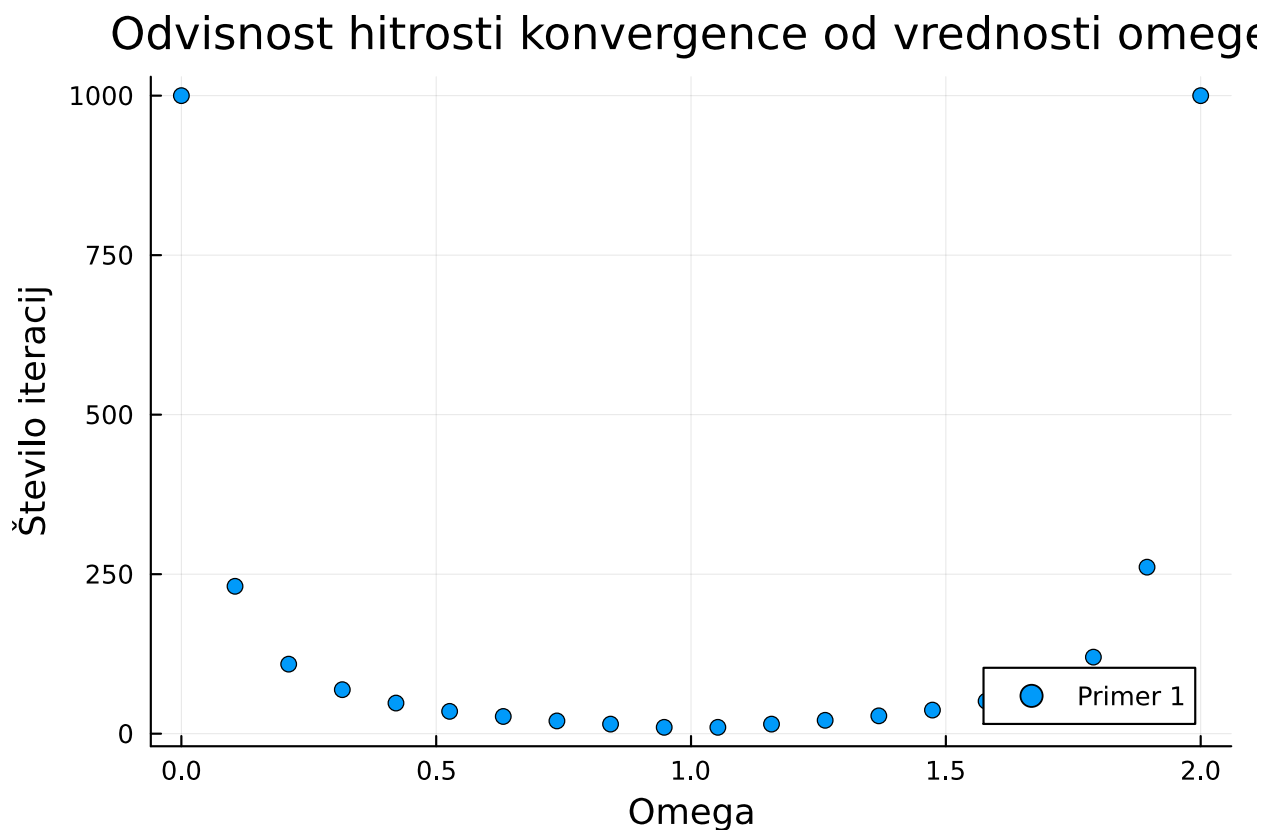
Rezultat

rez

```
3-element Vector{Float64}:  
 0.499999999999991845  
 3.33333333333523623  
 0.6666666666665579
```

Graf hitrosti konvergence

`display(plot)`



Primer 2

```
vozlisca = [1, 2, 3, 4]  
robovi = [(1, 1), (2, 2), (2, 3), (3, 3), (3, 1), (4, 3), (4, 4), (4, 2)]  
vrednosti = [2.0, 1.0, 1.0, 3.0, 1.0, 4.0, 3.0, 12.0]  
B = ustvariRazsprsenoMatriko(vozlisca, robovi, vrednosti)  
  
b = [1.0, 4.0, 2.0, 3.0]  
x0 = [0.0, 0.0, 0.0, 0.0]  
  
plot, minIteracij, minOmega, rez = optimalnaOmega(B, b, x0, 2)  
  
(Plot{Plots.GRBackend() n=2}, 10.0, 0.9473684210526315, [0.49999999999999184  
5, 3.4999999999932485, 0.5000000000000176, -13.666666666659667])
```

Optimalna vrednost omega

minOmega

0.9473684210526315

Minimalno število iteracij

minIteracij

10.0

Rezultat

rez

4-element Vector{Float64}:

0.499999999999991845

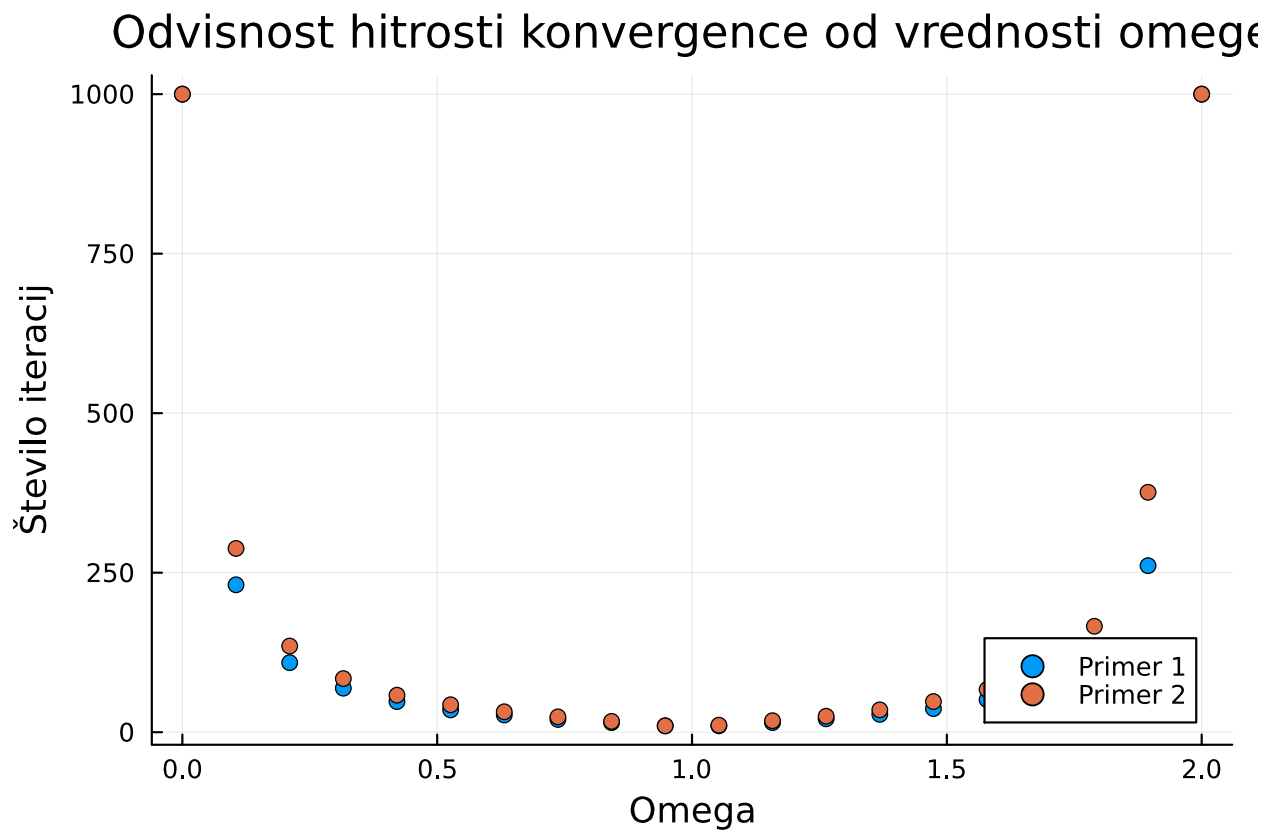
3.49999999999932485

0.50000000000000176

-13.6666666666659667

Graf hitrosti konvergence

`display(plot)`



Primer 3

`vozlisca = [1, 2, 3, 4]`

`robovi = [(1, 1), (2, 2), (2, 3), (3, 3), (3, 4), (3, 1), (4, 3), (4, 4), (4, 2), (4, 1)]`

`vrednosti = [2.0, 1.0, 1.0, 3.0, 1.0, 4.0, 3.0, 12.0, 1.5, 8.5]`

`C = ustvariRazsprsenoMatriko(vozlisca, robovi, vrednosti)`

`plot, minIteracij, minOmega, rez = optimalnaOmega(C, b, x0, 3)`

`(Plot{Plots.GRBackend() n=3}, 16.0, 0.9473684210526315, [0.5, 3.789855072529117, 0.21014492754805617, -0.6304347826215745])`

Optimalna vrednost omega

minOmega

0.9473684210526315

Minimalno število iteracij

minIteracij

16.0

Rezultat

rez

4-element Vector{Float64}:

0.5

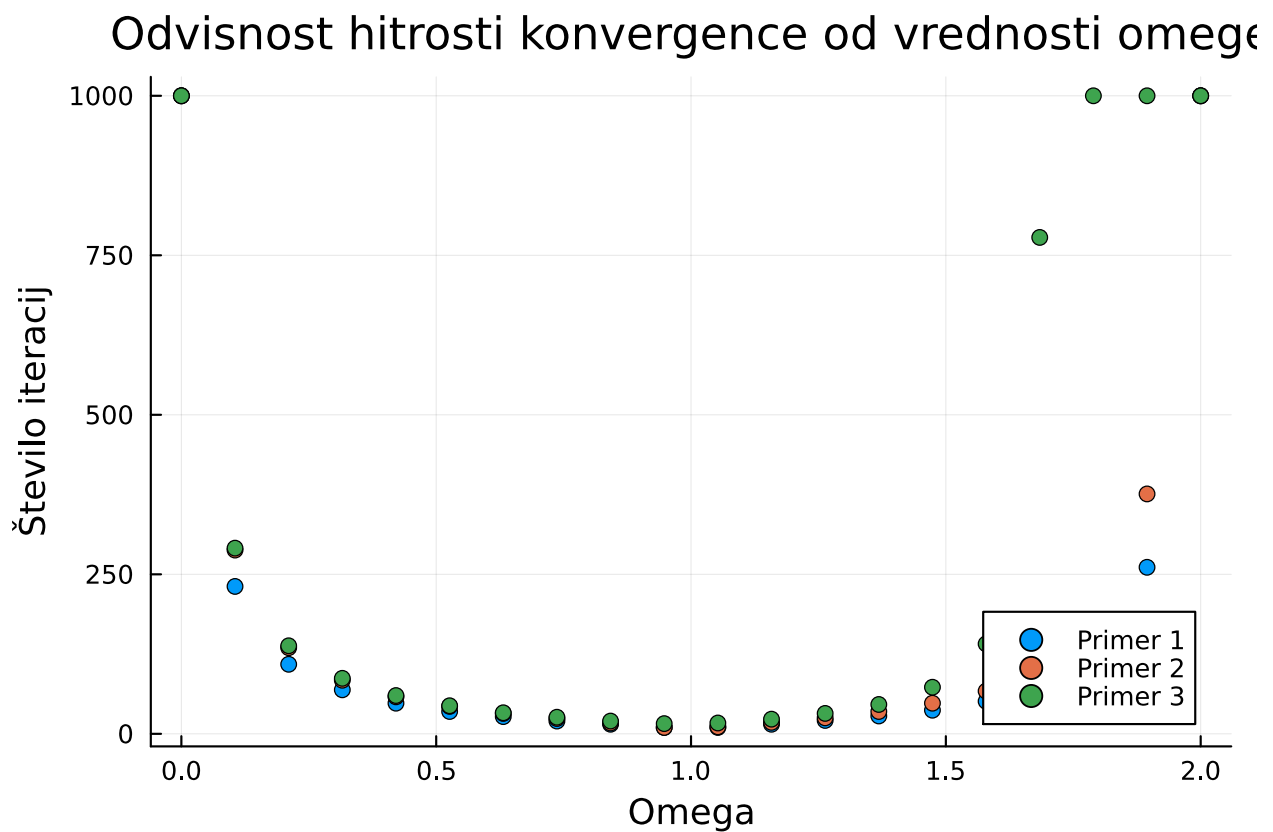
3.789855072529117

0.21014492754805617

-0.6304347826215745

Graf hitrosti konvergence

`display(plot)`



Primer 4: Vlaganje grafa z Laplaceovim sistemom in SOR iteracijo

```
vozlisca = [1, 2, 3, 4]
robovi = [(1,2), (2,3), (3,4)]
koordinate = Dict{
    1 => (-4.0, -5.0),
    4 => (2.0, 3.0)
}
```

```

Lred, bx, by = sistemLaplace(vozlisca, robovi, koordinate; fiksni=koordinate)

# Rešimo z uporabo lastne SOR metode
x0 = zeros(length(bx))
x, _ = SOR(Lred, bx, x0, 1.25)

y0 = zeros(length(by))
y, _ = SOR(Lred, by, y0, 1.25)

# Rekonstruiramo vse koordinate
free = setdiff(vozlisca, keys(koordinate))
for (i, vozlisce) in enumerate(free)
    koordinate[vozlisce] = (x[i], y[i])
end

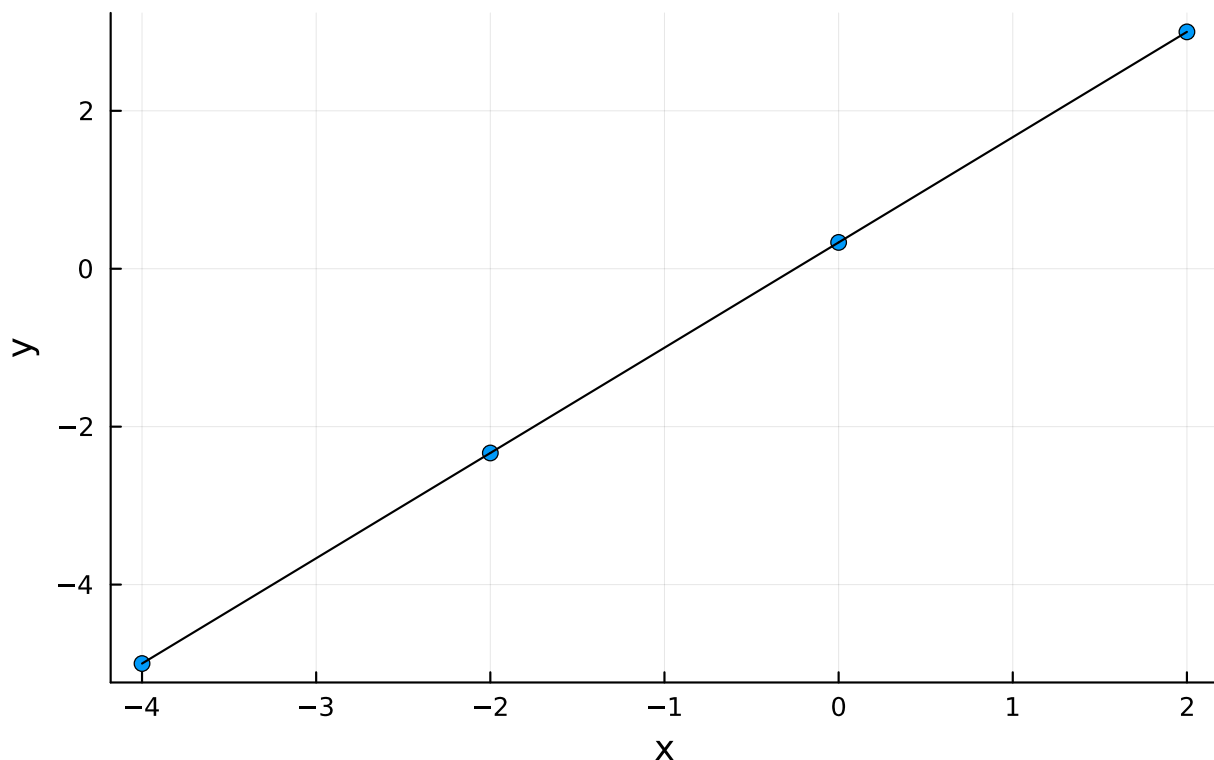
# Narišemo rezultat
gplot = scatter([p[1] for p in values(koordinate)],
                [p[2] for p in values(koordinate)],
                xlabel="x", ylabel="y", title="Vgrajevanje grafa z Laplaceovo metodo
(SOR)",
                label="vozlišča", legend=false)

for (i, j) in robovi
    xi, yi = koordinate[i]
    xj, yj = koordinate[j]
    plot!([xi, xj], [yi, yj], lw=1, color=:black)
end

display(gplot)

```

Vgrajevanje grafa z Laplaceovo metodo (SOR)



Primer 5: Še en primer grajevanja grafa z Laplaceovo metodo in SOR iteracijo

```

# Vozlišča in povezave (graf v obliki kvadrata z diagonalami)
vozlisca = [1, 2, 3, 4, 5]
robovi = [
    (1, 2), (2, 3), (3, 4), (4, 1), # kvadrat
    (1, 3), (2, 4),                # diagonali
    (1, 5), (2, 5), (3, 5), (4, 5) # center povezan z vsemi
]

# Fiksiramo oglišča kvadrata (1-4) v kotih
fiksne_koordinate = Dict{
    1 => (-2.0, -2.0),
    2 => (2.0, -2.0),
    3 => (2.0, 2.0),
    4 => (-2.0, 2.0)
}

# Pridobimo sistem iz Laplaceove matrike
Lred, bx, by = sistemLaplace(vozlisca, robovi, fiksne_koordinate;
fiksni=fiksne_koordinate)

# Rešimo z našo SOR metodo
x0 = zeros(length(bx))
x, _ = SOR(Lred, bx, x0, 1.25)

y0 = zeros(length(by))
y, _ = SOR(Lred, by, y0, 1.25)

# Združimo vse koordinate
koordinata = copy(fiksne_koordinate)
free_vozl = setdiff(vozlisca, keys(fiksne_koordinate))
for (i, v) in enumerate(free_vozl)
    koordinata[v] = (x[i], y[i])
end

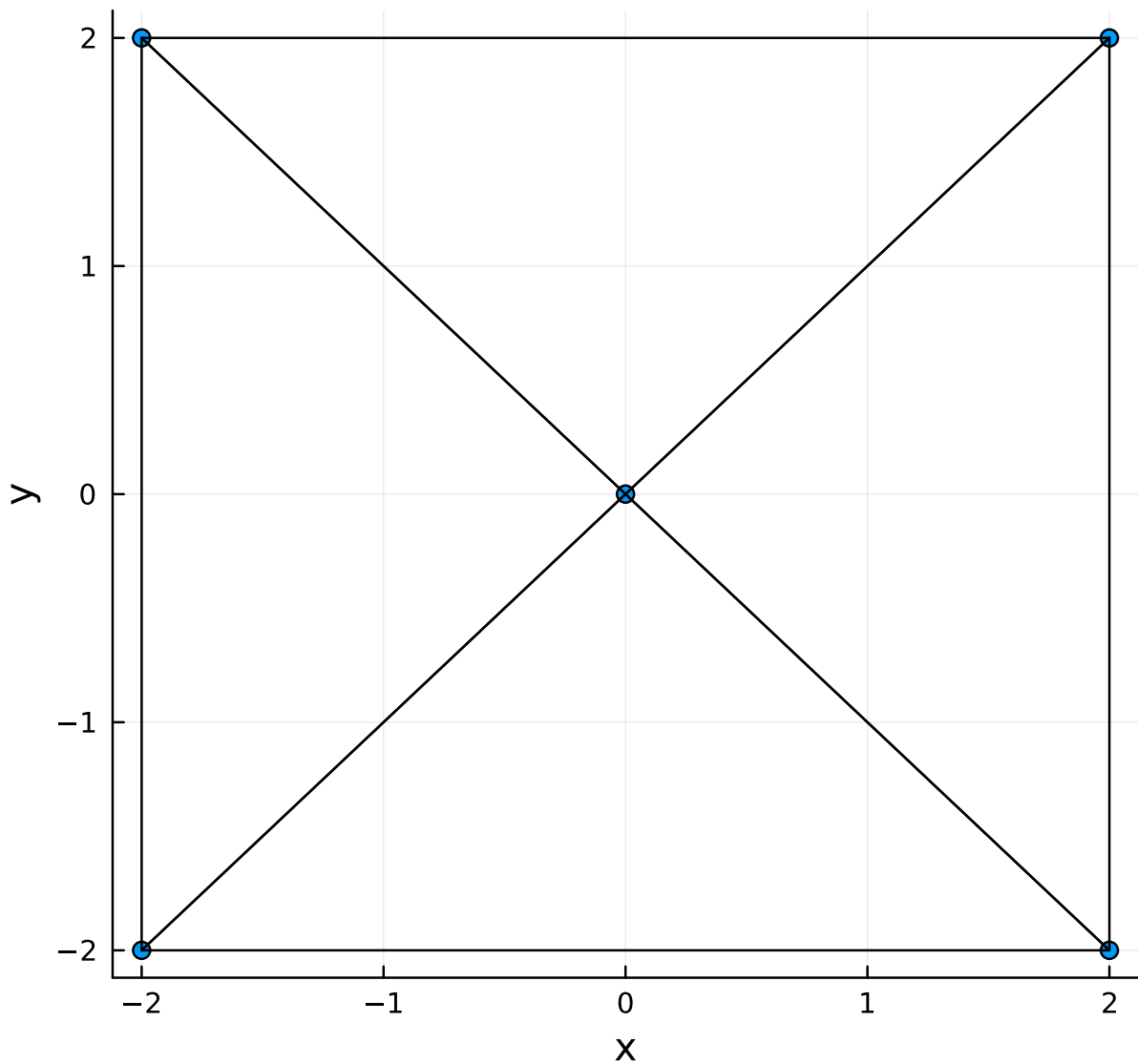
# Prikaz grafa
gplot = scatter(
    [p[1] for p in values(koordinata)],
    [p[2] for p in values(koordinata)],
    xlabel="x", ylabel="y", title="Laplaceovo vgrajevanje - Primer 5",
    label="vozlišča", legend=false, size=(500, 500)
)

for (i, j) in robovi
    xi, yi = koordinata[i]
    xj, yj = koordinata[j]
    plot!([xi, xj], [yi, yj], lw=1, color=:black)
end

display(gplot)

```

Laplaceovo vgrajevanje - Primer 5



1.2 Zaključek

Z uporabo metode SOR in optimizacije relaksacijskega faktorja smo učinkovito reševali sisteme z razpršenimi matrikami. Rezultati kažejo, da izbira primerne parametra ω pomembno vpliva na hitrost konvergence.